Car Wash App — Product Requirements Document

A project to build together, one working piece at a time.

1. What we're building

A web app for the car wash you work at. Customers can see the services and prices and book a slot. Staff can see a live queue of cars and move each one through the wash. The owner can see how many cars were done and how much was earned.

You already know this business better than anyone else who could build this. You know where things go wrong, so you decide what the app should do. I'll help with the how.

2. Why we're doing it

- **Learn the whole stack by building something real:** pages people see, a server behind them, a database that remembers things, logins, and a public link anyone can open.
- **Solve actual problems at the car wash:** double-booked slots, forgotten add-ons, confusion about which car is next.
- **End with something you can show:** a live project on your GitHub that you built and can explain, which is exactly what internship applications and interviews are about.

This is not a race. Each stage ends with something that works, and you move on when you're ready.

3. Ground rules

- **You type, I ask questions.** I won't grab the keyboard. If you're stuck, I'll ask questions that point you towards the answer.
- **No AI for writing code.** Documentation, Google, Stack Overflow and reading error messages are all fine and are part of the skill.
- **Everything goes through GitHub.** Work on a branch, open a pull request, and I'll review it like a real team would.
- **Small, regular sessions.** One to two hours, a couple of times a week. We stop on something that works.
- **Break things on purpose.** Bugs are where the learning happens. Nobody here is judged for not knowing something.

4. Who uses it

- **Customers:** want to know what's offered, how much it costs, and to book a time without calling.
- **Staff (you and your colleagues):** want to see which cars are waiting, in progress or finished.
- **Owner:** wants to see how busy the day was and how much was made.

5. Tech stack

We keep it small so you aren't learning ten tools at once.

- **Frontend:** HTML, CSS and plain JavaScript
- **Backend:** Java with Spring Boot, built with Maven
- **Database:** SQLite (moving to Postgres only if we deploy somewhere that needs it)
- **Version control:** Git and GitHub
- **Deployment:** decided in Stage 6

6. Stages

Each stage has a goal and a clear way of knowing it's done. We don't start the next one until the current one works.

Stage 1: The menu page

Goal: a good-looking static page for the car wash.

- Shows the list of services with prices and the opening hours
- Looks fine on a phone as well as a laptop
- **Done when:** you can open it in a browser and show it to your boss
- **You'll learn:** HTML, CSS, how a browser displays a page, Git basics

Stage 2: The price calculator

Goal: the page can work out a total.

- Pick a vehicle size and tick add-ons such as wax or interior
- The total updates instantly without reloading the page
- **Done when:** the prices match what the car wash actually charges
- **You'll learn:** JavaScript, variables, functions, handling clicks and inputs

Stage 3: Bookings that are saved

Goal: a booking survives a page refresh.

- A booking form (name, car, service, time slot)
- A Spring Boot server that receives the booking and saves it in a database
- A page that lists all bookings
- **Done when:** you can make a booking, close the browser, reopen it, and the booking is still there
- **You'll learn:** what a server is, HTTP requests, databases and SQL, designing tables yourself

Stage 4: Staff dashboard

Goal: staff can log in and manage the queue.

- Staff login (passwords stored safely, never as plain text)
- A live queue showing each car as Waiting, In progress or Done
- Staff can move a car between those states with a button
- **Done when:** two people can use it on two phones and see each other's changes
- **You'll learn:** authentication, sessions, security basics, keeping the screen in sync with the data

Stage 5: Smarter logic

Goal: the app prevents mistakes and gives useful numbers.

- No two bookings can take the same slot
- Estimated wait time for the next car
- Daily and weekly totals: cars washed and money earned
- **Done when:** you try to break it by double-booking and it refuses
- **You'll learn:** real problem-solving, edge cases, testing, and why algorithms matter when the problem is yours

Stage 6: Go live

Goal: a public link that other people can use.

- Deploy it online
- Ask the owner if the team can try it for a week
- Collect feedback and fix the top three problems
- **Done when:** a real customer or colleague uses it

- **You'll learn:** deployment, environments, and how real feedback shapes a product

7. First draft of the data

You'll design this properly yourself in Stage 3. As a starting point, think about:

- **Customers:** name, phone number
- **Services:** name, price, how long it takes
- **Bookings:** which customer, which car, which service, what time, current status
- **Staff:** name, login details

Don't treat this as fixed. If you think a table is missing, add it and tell me why.

8. Out of scope (for now)

These are not part of the first version. They are good ideas for later:

- Online payments
- Text or email reminders
- A mobile app
- Customer accounts and loyalty points
- Photos of the cars before and after

9. How we'll use GitHub

- One issue per task, grouped under the stage it belongs to
- Create a branch for each task, and open a pull request when it works
- A pull request description says what you changed and how you tested it
- Small pull requests are better than big ones
- Commit messages say what changed, such as "Add price calculator for vehicle sizes"

10. How we know it worked

- You can explain every part of the code you wrote
- Real people at the car wash have used it
- It's live, it's on your GitHub, and it has a README that says what it does and how to run it
- You're still enjoying it. If it stops being fun, we change the plan, not you.

11. Open questions for you

Write your answers down before Stage 1. There are no wrong answers.

- What annoys you most about how the car wash runs today?
- If you could only have one feature, which would it be?
- What should the app be called?
- Would the owner want to see this?